

Home Compass

A confidence-gated real-estate agent — from a browser tab to a ranked, sourced, guardrailed answer.

Two views cover the system: **the deployed services** — what actually runs, and what each piece talks to — and **the decision path** a single chat message takes once it reaches the agent. The second view is where this app differs from a generic chatbot: nothing gets searched until a deterministic score says there's enough to go on, and nothing ships until it's checked against a rulebook.

— live request path

--- offline / optional

● agent hub / decision points

● data at rest

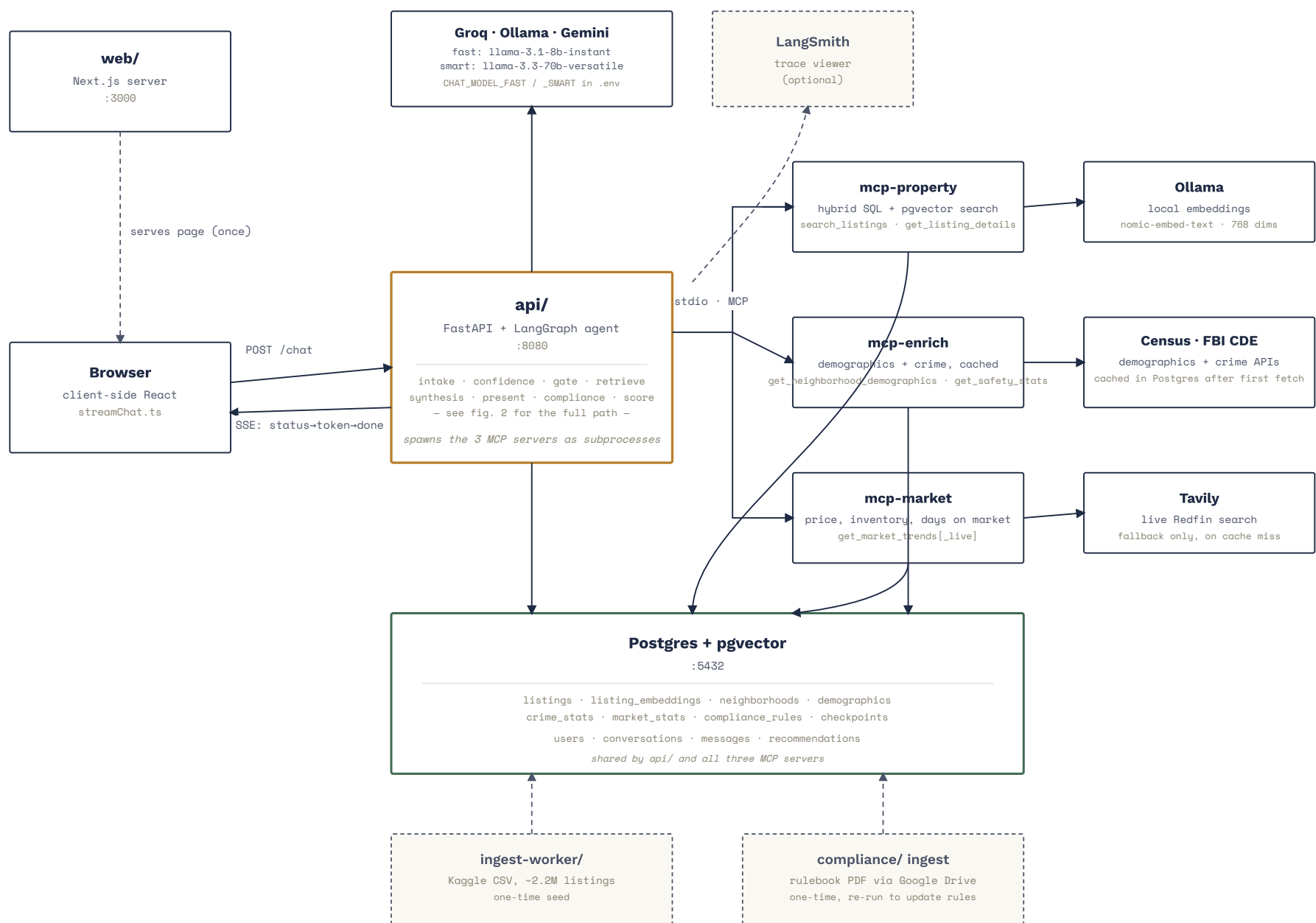


Fig. 1 The deployed system. A browser talks directly to `api/` — the Next.js server only serves the page once, then gets out of the way. `api/` spawns the three MCP tool servers as local subprocesses over stdio and holds the only LangGraph checkpointner; all four processes read and write the same Postgres database. The chat model itself is split into two roles by task, not one model for everything — see fig. 2.

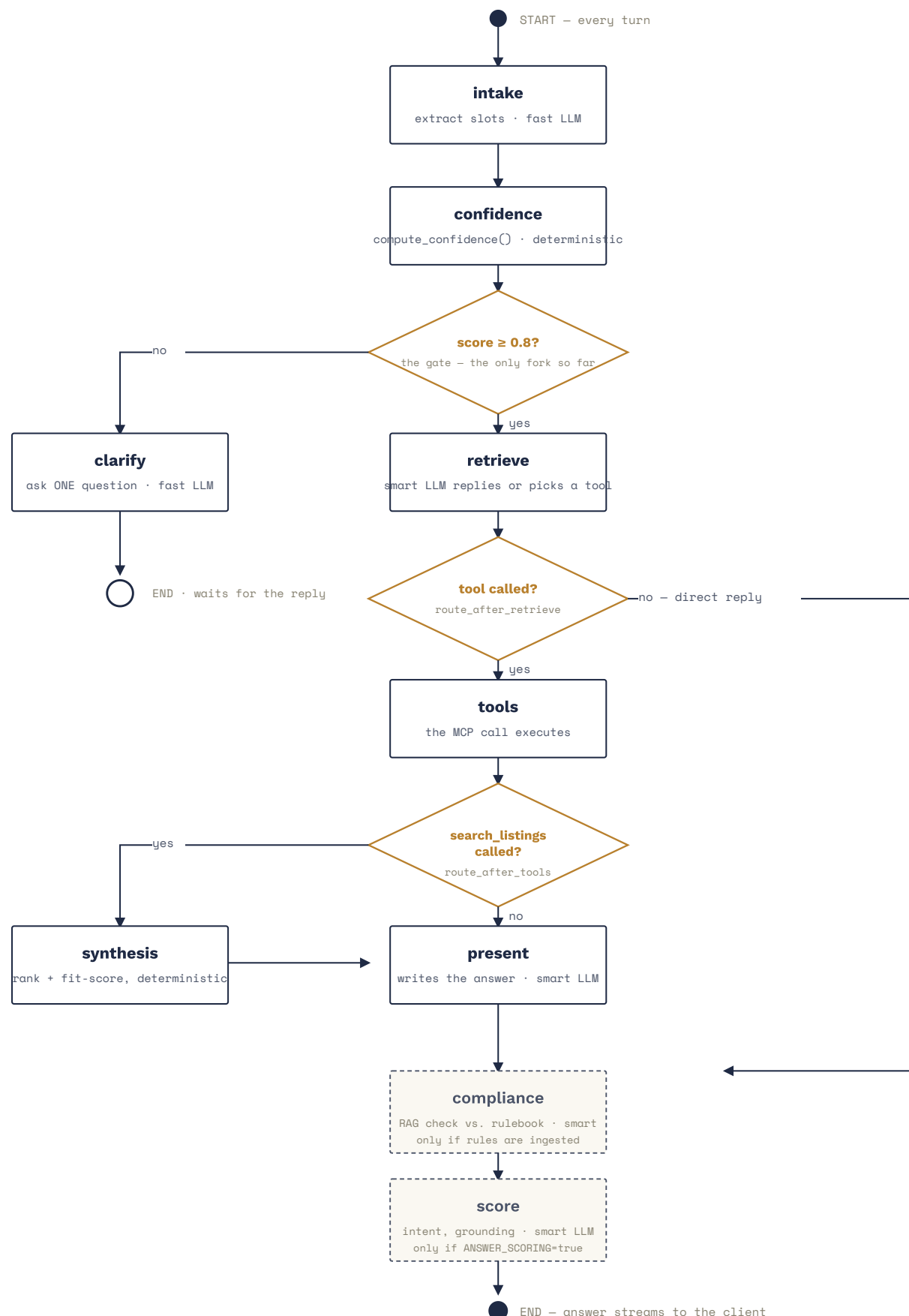


Fig. 2 One message's path through `api/agent.py`'s LangGraph state machine. The confidence gate (the compass reading shown throughout the conversation) is a plain deterministic function, not a model call — it has to be, since it's shown to the user and has to be reproducible. The chat model itself is split into two roles by task: a **fast**, cheap model handles `intake` and `clarify`, where a wrong call just means one more clarifying question; a **smart** model handles everywhere a wrong call is expensive — tool routing, the final answer, the compliance verdict, and the answer-grading judge in `score`. `compliance` and `score` are drawn dashed because either can be disabled at graph-build time; when both are off, every path above still ends the same way.